

Prompt:

8/17/2026, 10:22:31 PM

lanjut Tahap AG

Response:

8/17/2026, 10:22:32 PM

Tahap AG — Mixed-Integer Portfolio Optimization: dari "Trade mana?" menjadi "Trade mana + seberapa besar?"

Tahap AG adalah kelanjutan langsung dari AF. Di AF kita sudah mempunyai variabel biner

$$x_i \in \{0, 1\},$$

yang menjawab:

Apakah opportunity i dipilih?

Sekarang kita tambahkan variabel kontinu

$$w_i \in \mathbb{R},$$

yang menjawab:

Jika dipilih, seberapa besar modal/risk yang dialokasikan?

Maka sistem kita berubah dari **binary portfolio selection** menjadi:

Mixed-Integer Portfolio Optimization

dengan struktur fundamental:

$$x_i \in \{0, 1\}, \quad w_i \in \mathbb{R}.$$

AG.1. Dua variabel yang memiliki tugas berbeda

Untuk setiap opportunity i :

$$x_i = \begin{cases} 1, & \text{trade dipilih} \\ 0, & \text{trade tidak dipilih} \end{cases}$$

sedangkan:

$$w_i$$

menyatakan exposure atau position weight.

Keduanya harus dihubungkan.

Jika:

$$x_i = 0,$$

maka kita ingin:

$$w_i = 0.$$

Jika:

$$x_i = 1,$$

maka:

$$w_i$$

boleh mengambil nilai tertentu.

Secara matematis:

$$\boxed{|w_i| \leq M_i x_i}$$

dengan M_i sebagai maximum position size.

AG.2. Mengapa constraint ini sangat penting?

Tanpa constraint:

$$|w_i| \leq M_i x_i,$$

optimizer bisa menghasilkan:

$$x_i = 0, \quad w_i \in 0.$$

Artinya portfolio mengatakan:

"Saya tidak memilih trade ini, tetapi tetap memiliki posisi di dalamnya."

Tidak masuk akal.

Constraint tersebut membuat:

$$x_i = 0 \Rightarrow w_i = 0.$$

Sedangkan:

$$x_i = 1 \Rightarrow |w_i| \leq M_i.$$

AG.3. Portfolio return

Sekarang expected portfolio return menjadi:

$$E[R_p] = \sum_i w_i \mu_i$$

atau:

$$E[R_p] = \mathbf{w}^\top \boldsymbol{\mu}.$$

Berbeda dari AF.

Di AF:

$$\mathbf{x}^\top \boldsymbol{\mu}$$

menganggap setiap trade mempunyai kontribusi binary.

Sekarang:

$$w_i \mu_i$$

memperhitungkan **berapa besar exposure** yang kita berikan.

AG.4. Portfolio variance

Sekarang:

$$\sigma_p^2 = \mathbf{w}^\top \Sigma \mathbf{w}.$$

Sehingga objective Markowitz:

$$\max_{\mathbf{w}} [\mathbf{w}^\top \mu - \lambda \mathbf{w}^\top \Sigma \mathbf{w}].$$

Tetapi sekarang kita juga mempunyai x .

Maka:

$$\max_{\mathbf{x}, \mathbf{w}} [\mathbf{w}^\top \mu - \lambda \mathbf{w}^\top \Sigma \mathbf{w}].$$

subject to:

$$|w_i| \leq M_i x_i.$$

AG.5. Ini adalah inti mathematical architecture kita

Kita sekarang mempunyai:

x_i	selection
w_i	sizing

sehingga:

$$\text{Selection} + \text{Sizing} + \text{Risk}$$

dapat dioptimalkan secara simultan.

AG.6. Tetapi ada masalah besar

Bayangkan:

$$N = 1000.$$

Kita memiliki:

$$1000$$

variabel binary dan:

$$1000$$

variabel continuous.

Maka problem menjadi jauh lebih sulit daripada QUBO biasa.

Secara umum:

$$MIQP = \text{Mixed Integer Quadratic Programming.}$$

AG.7. Apakah kita harus langsung memasukkan semuanya ke QAOA?

Menurut saya:

tidak.

Ini keputusan desain yang penting.

Kita jangan memaksakan:

semua matematika $\rightarrow$ quantum.

Justru kita gunakan paradigma hybrid:

Quantum + Classical

sesuai struktur masalah.

AG.8. Hierarchical decomposition

Kita dapat memecahkan:

$$(x, w)$$

menjadi dua tahap:

Tahap 1

Cari:

$$x^*.$$

Tahap 2

Diberikan x^* , cari:

$$w^*.$$

Secara matematis:

$$x^* = \arg \min_x H(x)$$

kemudian:

$$w^* = \arg \min_w f(w | x^*).$$

AG.9. Mengapa pendekatan ini sangat masuk akal?

Karena discrete problem menjawab:

"Apa yang kita percaya layak diperdagangkan?"

Sedangkan continuous problem menjawab:

"Bagaimana kita mengalokasikan risk?"

Dua pertanyaan tersebut memang berbeda secara konseptual.

AG.10. Position sizing berbasis stop-loss

Sekarang kita hubungkan dengan sistem \$ \$1 \$ risk yang kita desain sebelumnya.

Misalkan entry:

$$P_i^{entry}$$

dan stop-loss:

$$P_i^{SL}.$$

Distance:

$$d_i = |P_i^{entry} - P_i^{SL}|.$$

Untuk satu unit posisi, maximum loss:

$$L_i = d_i V_i,$$

dengan V_i adalah nilai moneter per unit perubahan harga.

Jika risk budget:

$$R_i^{max} = \$1,$$

maka:

$$q_i^{max} = \frac{1}{d_i V_i}.$$

AG.11. Jika portfolio menggunakan risk unit

Lebih fleksibel jika kita mendefinisikan:

$$r_i$$

sebagai risk budget trade dalam satuan R .

Misalnya:

$$r_i = 1R.$$

Kemudian:

$$q_i = \frac{r_i R_{portfolio}}{d_i V_i}.$$

Jadi:

$$Position\ Size = \frac{Risk\ Budget}{Stop\ Distance \times Value\ per\ Unit}.$$

Ini adalah fondasi risk-based position sizing.

AG.12. Sekarang w_i dapat kita definisikan dalam risk-space

Daripada:

$$w_i = \frac{notional_i}{Capital},$$

kita bisa mendefinisikan:

$$w_i = \text{fraction of portfolio risk.}$$

Misalnya:

$$w_{BTC} = 0.15$$

berarti BTC menerima:

$$15\%$$

dari risk budget tertentu.

Ini jauh lebih konsisten dengan sistem trading kita daripada sekadar equal capital weighting.

AG.13. Risk contribution

Untuk portfolio:

$$\sigma_p = \sqrt{\mathbf{w}^\top \Sigma \mathbf{w}},$$

marginal contribution to risk dari asset i :

$$MCR_i = \frac{\partial \sigma_p}{\partial w_i}.$$

Karena:

$$\sigma_p = \sqrt{\mathbf{w}^\top \Sigma \mathbf{w}},$$

maka:

$$\frac{\partial \sigma_p}{\partial w_i} = \frac{(\Sigma \mathbf{w})_i}{\sigma_p}.$$

Sehingga:

$$MCR_i = \frac{(\Sigma \mathbf{w})_i}{\sigma_p}.$$

AG.14. Total risk contribution

Risk contribution:

$$RC_i = w_i MCR_i.$$

Maka:

$$RC_i = \frac{w_i (\Sigma \mathbf{w})_i}{\sigma_p}.$$

Dan:

$$\sum_i RC_i = \sigma_p.$$

Ini sangat berguna untuk mengetahui:

Trade mana yang sebenarnya menyumbang risiko terbesar?

Karena position terbesar belum tentu mempunyai risk contribution terbesar.

AG.15. Contoh

Misalkan:

$$w_A = 0.2$$

dan:

$$w_B = 0.1.$$

Tetapi A dan B sangat berkorelasi.

Mungkin:

$$RC_A = 0.15$$

dan:

$$RC_B = 0.12.$$

Jadi B yang hanya setengah posisi A ternyata menyumbang risiko hampir sama.

Inilah alasan:

$$\text{Position Weight} \propto \text{Risk Weight}.$$

AG.16. Risk parity sebagai alternatif

Kita dapat meminta:

$$RC_i \approx RC_j.$$

Untuk seluruh trade:

$$RC_i \approx \frac{\sigma_p}{N}.$$



Ini disebut **risk parity**.

Tetapi untuk sistem kita, saya tidak menyarankan risk parity sebagai objective utama.

Lebih baik menjadikannya:

$$\text{optional regularizer.}$$

Karena kita tetap ingin mempertahankan expected edge.

AG.17. Objective sizing yang lebih lengkap

Setelah x^* ditentukan:

$$\max_w [w^\top \mu - \lambda w^\top \Sigma w]$$

subject to:

$$|w_i| \leq M_i x_i^*,$$

$$\sum_i |w_i| \leq L_{\max},$$

dan:

$$CVaR(w) \leq C_{\max}.$$

AG.18. Leverage constraint

Jika portfolio mempunyai leverage maksimum:

$$L_{\max},$$

maka:

$$\sum_i |w_i| \leq L_{\max}.$$

Misalnya:

$$L_{\max} = 2.$$

Maka total gross exposure tidak boleh melebihi:

$$2 \times \text{capital}.$$

AG.19. Long-short portfolio

Untuk sistem yang mendukung short:

$$w_i \in [-M_i, M_i].$$

Kita dapat memecah:

$$w_i = w_i^+ - w_i^-,$$

dengan:

$$w_i^+ \geq 0,$$

$$w_i^- \geq 0.$$

Dan:

$$|w_i| = w_i^+ + w_i^-.$$

AG.20. Constraint selection-sizing

Kita dapat menggunakan:

$$w_i^+ + w_i^- \leq M_i x_i.$$

Sehingga jika:

$$x_i = 0 :$$

$$w_i^+ = w_i^- = 0.$$

Jika:

$$x_i = 1 :$$

posisi dapat long atau short.

AG.21. Factor neutrality

Misalkan:

$$B$$

adalah factor loading matrix.

Portfolio exposure:

$$f = B^T w.$$

Jika kita ingin market-neutral:

$$B_{market}^T w = 0.$$

Jika kita ingin USD-neutral:

$$B_{USD}^T w = 0.$$

Atau hanya membatasi:

$$|B_{USD}^T w| \leq \epsilon_{USD}.$$

AG.22. Ini sangat relevan dengan desain kita

Misalnya model menemukan:

$$10$$

saham yang semuanya bullish.

Tetapi ternyata semuanya mempunyai:

$$\beta_{USD} < 0.$$

Artinya portfolio sebenarnya memiliki satu hidden macro bet:

$$\boxed{\text{Short USD.}}$$

Kita bisa mengontrolnya secara eksplisit.

AG.23. Gold factor

Hal yang sama berlaku untuk gold.

Jika:

$$B_{Gold}^T w$$

terlalu besar, berarti portfolio mempunyai exposure besar terhadap factor gold.

Kita bisa:

$$B_{Gold}^T w \leq B_{Gold}^{max}.$$

Atau bahkan:

$$B_{Gold}^T w \approx 0.$$

AG.24. Faktor bukan hanya untuk risk

Ini penting.

Factor exposure juga bisa menjadi **signal**.

Misalnya:

$$F_{USD} \uparrow$$

dan model kita memperkirakan aset tertentu akan underperform.

Maka:

$$\mu_i$$

dapat dibuat conditional:

$$\mu_i = E[R_i | F_t].$$

Dengan demikian factor model masuk ke:

$$Prediction$$

dan:

$$Risk.$$

AG.25. Conditional covariance

Lebih lanjut:

$$\Sigma_t$$

juga tidak harus konstan.

Kita dapat memiliki:

$$\Sigma_t = B_t \Sigma_{F,t} B_t^\top + D_t.$$

Jadi ketika volatility regime berubah:

$$\Sigma_{F,t}$$

ikut berubah.

AG.26. Regime-dependent optimization

Misalkan terdapat regime:

$$z_t \in \{Bull, Bear, HighVol, LowVol, RiskOn, RiskOff\}.$$

Maka:

$$\mu_t = \mu(z_t)$$



dan:

$$\Sigma_t = \Sigma(z_t).$$

Optimizer menjadi:

$$w_t^* = w^*(z_t).$$

AG.27. Ini membuat sistem adaptif

Contohnya:

Risk-on

$$F_{crypto} > 0$$

$$F_{equity} > 0$$

volatility rendah.

Optimizer mungkin memperbolehkan:

higher gross exposure.

Risk-off

Volatility meningkat.

Optimizer mengurangi:

$$\|w\|_1.$$

AG.28. Dynamic risk budget

Bahkan:

$$R_{max}$$

dapat dibuat fungsi regime:

$$R_{max,t} = R_0 g(z_t).$$

Misalnya:

$$g(z_t) < 1$$

ketika:

volatility

sangat tinggi.

Ini berarti sistem otomatis melakukan **risk compression**.

AG.29. Sekarang kita kembali ke \$1 risk

Ada dua level:

Trade level

$$Risk_i \leq \$1.$$

Portfolio level

$$Risk_p \leq R_{portfolio}^{max}.$$

Keduanya harus dipertahankan.

Misalnya:

$$R_i = \$1$$

untuk setiap trade.

Jika:

$$R_{portfolio}^{max} = \$5,$$

maka sistem tidak boleh membuka 20 trade independen sekaligus jika aggregate risk melebihi budget.

AG.30. Correlation-adjusted risk

Worst-case:

$$R_p^{worst} = \sum_i R_i.$$

Tetapi expected aggregate risk lebih kecil jika trade independen.

Secara sederhana:

$$V ar(L) = \sum_i \sigma_i^2 + 2 \sum_{i < j} \rho_{ij} \sigma_i \sigma_j.$$

Inilah alasan correlation matters.

AG.31. Bahkan risk \$1 dapat dibuat adaptive

Tidak harus selalu:

$$\$1.$$

Kita dapat mendefinisikan:

$$R_i(t) = R_0 g(EV_i, Vol_t, Liquidity_t, Drawdown_t).$$

Misalnya:

$$R_i(t) = \$1$$

dalam kondisi normal.

Tetapi ketika drawdown meningkat:

$$R_i(t) < \$1.$$

AG.32. Drawdown-aware risk scaling

Misalkan equity:

$$E_t.$$



High-water mark:

$$H_t = \max_{\tau \leq t} E_\tau.$$

Drawdown:

$$DD_t = 1 - \frac{E_t}{H_t}.$$

Kemudian:

$$R_t = R_0 f(DD_t).$$

Contoh:

$$f(DD) = \max(f_{\min}, 1 - kDD).$$

Jadi semakin besar drawdown:

$$\text{risk budget} \downarrow.$$

AG.33. Ini merupakan feedback control

Kita sekarang memiliki:

$$\text{Market} \rightarrow \text{Signal} \rightarrow \text{Portfolio} \rightarrow \text{PnL} \rightarrow \text{Drawdown} \rightarrow \text{Risk Budget} \rightarrow \text{Portfolio}.$$

Dengan demikian sistem menjadi:

$$\text{closed loop}.$$

AG.34. Transaction cost

Sizing tidak boleh mengabaikan biaya.

Misalkan:

$$C_i$$

adalah expected transaction cost per unit exposure.

Maka:

$$Cost(w) = \sum_i c_i |w_i|.$$

Objective:

$$\max [w^T \mu - \lambda w^T \Sigma w - c^T |w|].$$

AG.35. Market impact

Untuk trade yang lebih besar, cost tidak lagi linear.

Model sederhana:

$$Impact_i(q_i) = \eta_i q_i^2.$$

Maka:

$$Cost_i(q_i) = c_i |q_i| + \eta_i q_i^2.$$

Ini membuat optimizer memperhitungkan liquidity.

AG.36. Mengapa ini penting untuk short-term trading?

Karena pada horizon pendek:

$$Expected\ Edge$$

bisa sangat kecil.

Misalnya:

$$EV = 0.15R.$$

Tetapi:

$$Transaction\ Cost = 0.08R.$$

Maka net edge:

$$0.07R.$$

Jika slippage:

$$0.10R,$$

trade tersebut sebenarnya:

$$EV_{net} < 0.$$

AG.37. Maka expected value kita harus diperbaiki

Jangan menggunakan:

$$EV_i^{gross}.$$

Gunakan:

$$EV_i^{net} = EV_i^{gross} - Cost_i - Slippage_i - Funding_i.$$

Untuk crypto:

$$Funding_i$$

bisa penting.

Untuk saham:

$$BorrowCost_i$$

dapat relevan jika short.

AG.38. Objective final sizing

Kita sekarang dapat menulis:

$$\max_{\mathbf{w}} \quad \mathbf{w}^\top \boldsymbol{\mu} - \lambda \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w} \\ - \mathbf{c}^\top |\mathbf{w}| - \mathbf{w}^\top \mathbf{M} \mathbf{w}$$

subject to:

$$|w_i| \leq M_i x_i^*,$$

$$\sum_i |w_i| \leq L_{\max},$$

$$B^\top \mathbf{w} \in \mathcal{F},$$

$$CVaR(\mathbf{w}) \leq C_{\max}.$$

AG.39. Interpretasi setiap term

$$\mathbf{w}^\top \boldsymbol{\mu}$$

= expected profit.

$$\lambda \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$$

= portfolio risk.

$$\mathbf{c}^\top |\mathbf{w}|$$

= transaction cost.

$$\mathbf{w}^\top \mathbf{M} \mathbf{w}$$

= market impact.

Constraint:

$$|w_i| \leq M_i x_i$$

= selection control.

Constraint:

$$B^T w \in F$$

= factor exposure control.

AG.40. Sekarang kita mempunyai dua optimizer

Optimizer A

Discrete Optimizer

memilih:

$$x^*.$$

Optimizer B

Continuous Optimizer

memilih:

$$w^*.$$

AG.41. Tetapi kita bisa melakukan sesuatu yang lebih menarik

Setelah mendapatkan:

$$x^*,$$

kita dapat melakukan **iterative refinement**.

Pertama:

$$x^{(0)} \rightarrow w^{(0)}.$$

Kemudian hitung:

$$Risk(w^{(0)}).$$

Jika terlalu tinggi, kita kembali ke selection:

$$x^{(1)}.$$

Lalu:

$$x^{(1)} \rightarrow w^{(1)}.$$

Dan seterusnya.

AG.42. Jadi bukan hanya pipeline

Bukan:

QUBO → Sizing → selesai

tetapi:



AG.43. Ini merupakan decomposition algorithm

Secara konseptual:

$$\boxed{Discrete \rightarrow Continuous \rightarrow Risk \rightarrow Discrete \rightarrow \dots}$$

Kita bisa menggunakan pendekatan seperti:

- alternating optimization,
- Benders-style decomposition,
- coordinate refinement,
- heuristic local search.

Tidak semuanya harus quantum.

AG.44. Peran QAOA menjadi semakin jelas

QAOA tidak perlu mengoptimalkan:

$$w_i$$

secara langsung.

Ia dapat mengoptimalkan:

$$\boxed{x_i}.$$

Kemudian classical optimizer menangani:

$$\boxed{w_i}.$$

Ini membuat penggunaan QAOA jauh lebih realistis.

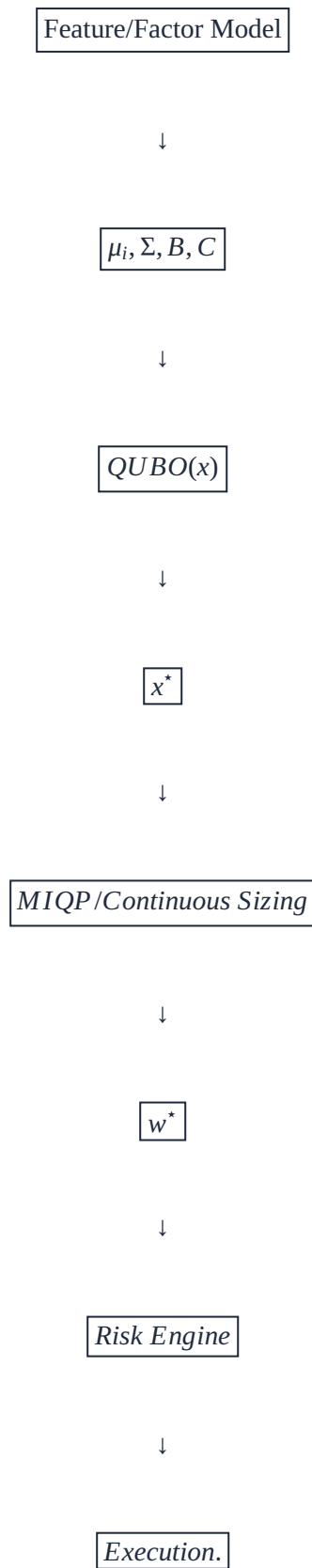
AG.45. Pipeline matematis Tahap AG

Sekarang sistem kita dapat dirumuskan:

$$\boxed{\text{Market Data}}$$

↓





AG.46. Contoh numerik kecil

Misalkan ada tiga trade:

A, B, C .

Expected return:

$$\mu = \begin{bmatrix} 0.12 \\ 0.10 \\ 0.08 \end{bmatrix}.$$

Covariance:

$$\Sigma = \begin{bmatrix} 0.04 & 0.03 & 0.00 \\ 0.03 & 0.05 & 0.01 \\ 0.00 & 0.01 & 0.02 \end{bmatrix}.$$

Misalkan optimizer memilih:

$$x^* = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

Kemudian sizing mencari:

$$w^*.$$

Dengan constraint:

$$w_A + w_B + w_C = 1.$$

dan:

$$w_i \geq 0.$$

AG.47. Unconstrained Markowitz

Tanpa constraint lain, problem:

$$\max_w [w^T \mu - \lambda w^T \Sigma w].$$

First-order condition:

$$\nabla (w^T \mu - \lambda w^T \Sigma w) = 0.$$

Karena:

$$\nabla(w^\top \mu) = \mu$$

dan:

$$\nabla(w^\top \Sigma w) = 2\Sigma w,$$

maka:

$$\mu - 2\lambda \Sigma w = 0.$$

Sehingga:

$$w^* = \frac{1}{2\lambda} \Sigma^{-1} \mu.$$

Ini adalah solusi dasar Markowitz.

AG.48. Tetapi constraint budget mengubahnya

Jika:

$$\mathbf{1}^\top w = 1,$$

kita gunakan Lagrange multiplier:

$$L = w^\top \mu - \lambda w^\top \Sigma w + v(1 - \mathbf{1}^\top w).$$

FOC:

$$\mu - 2\lambda \Sigma w - v\mathbf{1} = 0.$$

Sehingga:

$$w = \frac{1}{2\lambda} \Sigma^{-1} (\mu - v\mathbf{1}).$$

v kemudian ditentukan dari:

$$\mathbf{1}^\top w = 1.$$



AG.49. Kenapa derivasi ini penting?

Karena kita sekarang memiliki pembagian kerja:

Quantum/discrete

Menentukan:

$$x.$$

Classical analytical/numerical

Menentukan:

$$w.$$

Dengan demikian kita dapat menggunakan matematika yang sudah kamu pelajari dalam Markowitz secara langsung.

AG.50. Tetapi untuk trading jangka pendek, Markowitz saja belum cukup

Karena:

$$\mu_i$$

sangat noisy.

Maka kita tidak boleh memperlakukan:

$$\hat{\mu}_i$$

seolah-olah exact.

Kita harus memperhitungkan uncertainty:

$$\mu_i = \hat{\mu}_i + \epsilon_i.$$

AG.51. Robust optimization

Salah satu pendekatan:

$$\mu_i \in U_i.$$

Kita tidak mengoptimalkan:

$$\hat{\mu}_i$$

saja, tetapi worst-case expected return:

$$\max_w \min_{\mu \in U} w^T \mu - \lambda w^T \Sigma w.$$

Ini jauh lebih masuk akal untuk short-horizon prediction.

AG.52. Confidence-adjusted expected return

Alternatif sederhana:

$$\mu_i^{adj} = \hat{\mu}_i - \kappa S_i$$

dengan:

$$S_i$$

sebagai uncertainty estimate.

Jadi:

$$\text{high prediction uncertainty} \Rightarrow \mu_i^{adj} \downarrow.$$

AG.53. Ini sangat penting bagi sistem kita

Sebuah trade bisa memiliki:

$$\hat{\mu}_i = 0.20R$$

tetapi uncertainty:

$$s_i = 0.30R.$$

Trade tersebut sebenarnya kurang menarik dibanding trade:

$$\mu_j = 0.15R, \quad s_j = 0.05R.$$

Maka:

$$\mu_i^{adj} < \mu_j^{adj}.$$

AG.54. Confidence masuk ke QUBO

Kita sekarang dapat mengganti:

$$EV_i$$

dengan:

$$EV_i^{robust}.$$

Sehingga QUBO memilih bukan:

trade dengan predicted return terbesar,

tetapi:

trade dengan risk-adjusted, uncertainty-adjusted edge terbesar.

AG.55. Ini menghasilkan objective yang lebih matang

Discrete stage:

$$\begin{aligned} \min_x \quad & - \sum_i EV_i^{robust} x_i \\ & + \lambda x^\top \Sigma x \\ & + \lambda_F R_F(x) \\ & + \lambda_T T(x, x_{t-1}) \\ & + P_K(\mathbf{1}^\top x - K)^2. \end{aligned}$$

Continuous stage:

$$\begin{aligned} \max_w \quad & w^\top \mu^{robust} - \lambda w^\top \Sigma w \\ & - Cost(w) - Impact(w) \end{aligned}$$

subject to risk and exposure constraints.

AG.56. Satu lagi masalah: turnover

Misalkan:

$$w_{t-1}$$

adalah portfolio sebelumnya.

Maka perubahan:

$$\Delta w_t = w_t - w_{t-1}.$$

Transaction cost:

$$Cost_t = c^\top |\Delta w_t|.$$

Jadi objective sebenarnya harus menggunakan:

$$w_t^\top \mu_t - \lambda w_t^\top \Sigma_t w_t - c^\top |w_t - w_{t-1}|.$$

Ini membuat portfolio **sticky**.

AG.57. Ini sangat cocok dengan sistem evaluasi periodik kita

Kamu sebelumnya menginginkan:

setiap jam atau periode tertentu posisi dievaluasi.

Sekarang kita punya formulasi matematisnya.

Pada waktu t :

$$w_{t-1}$$

adalah portfolio saat ini.

Model menghitung:

$$w_t^*$$

Kemudian:

$$\Delta w_t = w_t^* - w_{t-1}.$$

Jika:

$$|\Delta w_t|$$

kecil, kita tidak perlu banyak melakukan perubahan.

AG.58. Bahkan kita bisa membuat minimum improvement threshold

Misalkan:

$$U(w_t^*) - U(w_{t-1}) < \epsilon.$$

Maka:

Do not rebalance.

Ini sangat penting agar optimizer tidak overtrade.

AG.59. Decision rule

Secara sederhana:

$$\Delta U = U(w_t^*) - U(w_{t-1}).$$

Jika:

$$\Delta U > \epsilon,$$

rebalancing.

Jika:

$$\Delta U \leq \epsilon,$$

pertahankan portfolio.

AG.60. Ini menghubungkan AG dengan sistem position management sebelumnya

Kita sebelumnya membicarakan:

- floating loss,
- floating profit,
- BEP,
- trailing SL,
- TP extension.

Sekarang kita dapat melihatnya sebagai:

$$\textit{Position Management} = \textit{Local Optimization}$$

sedangkan:

$$\textit{Portfolio Selection} = \textit{Global Optimization}.$$

AG.61. Dua skala optimisasi

Global

Which trades?

→ *QUBO*.

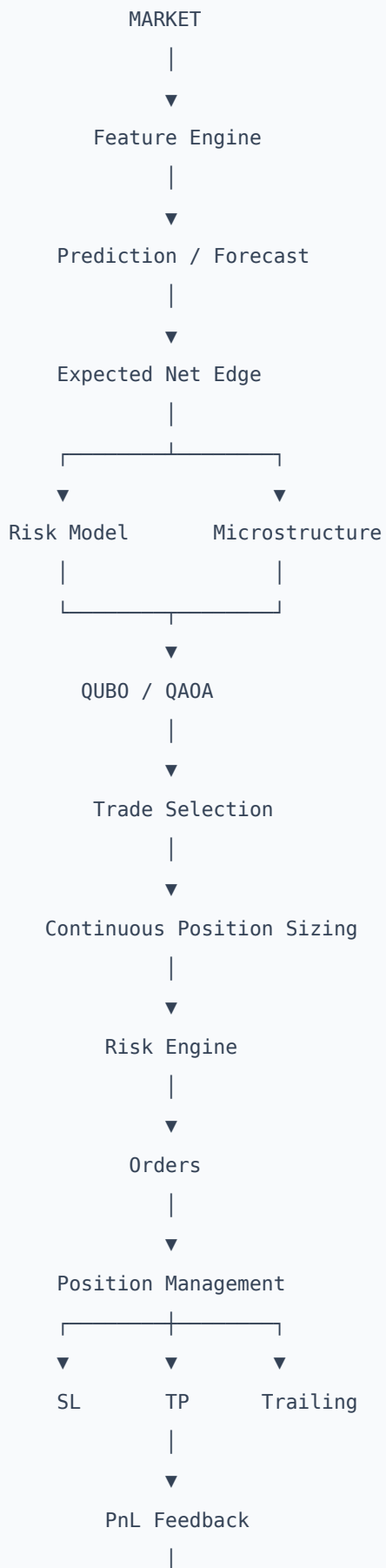
Local

How should each position evolve?

→ *SL/TP/Trailing*.

Jangan dicampur menjadi satu optimizer raksasa.

AG.62. Arsitektur akhirnya mulai terlihat



AG.63. Poin terpenting Tahap AG

Kita sekarang mempunyai **tiga lapisan optimisasi**:

Layer 1	:	Opportunity Prediction
Layer 2	:	Discrete Portfolio Selection
Layer 3	:	Continuous Position Sizing

dan kemudian:

Layer 4: Dynamic Position Management.

AG.64. Hubungan dengan TSP, Max-Cut, dan problem combinatorial lainnya

Sekarang menjadi lebih jelas mengapa kamu sejak awal tertarik pada:

- TSP,
- Max-Cut,
- portfolio optimization.

Mereka tidak harus menjadi tiga algoritma yang berdiri sendiri.

Mereka merupakan **primitive optimization structures**.

Misalnya:

<i>Portfolio Selection</i> → <i>QUBO</i>
--

<i>Diversification/Clustering</i> → <i>Graph/MaxCut</i>

dan untuk masalah execution:

<i>Order Routing</i> → <i>TSP/VRP</i>

AG.65. Bahkan execution bisa menjadi optimization problem

Misalnya kita memiliki:

N

venue/order types.

Kita ingin menentukan:

order sequence

dengan:

- latency,
- fee,
- liquidity,
- slippage.

Maka kita dapat memiliki:

Execution Optimization.

Ini adalah layer yang berbeda dari portfolio selection.

AG.66. Maka sistem besar kita mulai menjadi

Prediction → Portfolio → Execution → Position Management.

Dengan berbagai mathematical optimization primitive:

ML + QUBO + Ising + QAOA + Markowitz + Graph Optimization + Control.

AG.67. Dan ada prinsip desain yang sangat penting

Jangan menggunakan quantum algorithm hanya karena:

"masalahnya bisa dibuat QUBO."

Pertanyaan yang benar adalah:

Apakah quantum optimizer memberikan keuntungan dibanding classical baseline?

Karena sistem kita harus mempunyai benchmark:

QAOA

vs.

Simulated Annealing

vs.

MILP/MIQP

vs.

Greedy

vs.

Classical Markowitz.

AG.68. Ini nantinya akan menjadi eksperimen yang sangat kuat

Untuk setiap periode:

t ,

kita dapat menjalankan:

x_t^{QAOA}

dan:

$x_t^{classical}$.

Kemudian bandingkan:

EV,

Sharpe,

MaxDD,

CVaR,

Turnover,

Transaction Cost.

Dengan demikian kita tidak mengatakan:

"QAOA lebih bagus karena quantum."

Tetapi:

"Pada problem dan constraint tertentu, QAOA menghasilkan solusi dengan kualitas X dan computational characteristic Y dibanding baseline Z."

Ini jauh lebih ilmiah.

AG.69. Rumusan matematis AG yang paling lengkap

Untuk sekarang, saya akan menjadikan formulasi berikut sebagai **master formulation**:

$$\begin{aligned}
& \max_{\mathbf{x}, \mathbf{w}} \quad \mathbf{w}^\top \mu_t^{robust} - \lambda \mathbf{w}^\top \Sigma_t \mathbf{w} \\
& \quad - C_t(\mathbf{w} - \mathbf{w}_{t-1}) - I_t(\mathbf{w}) \\
& \quad - \lambda_F \|B_t^\top \mathbf{w}\|^2 \\
& \text{s.t.} \quad |w_i| \leq M_i x_i, \\
& \quad x_i \in \{0, 1\}, \\
& \quad \sum_i x_i \leq K, \\
& \quad \sum_i |w_i| \leq L_{\max}, \\
& \quad CVaR_q(\mathbf{w}) \leq C_{\max}, \\
& \quad B_t^\top \mathbf{w} \in F, \\
& \quad R_{portfolio}(\mathbf{w}) \leq R_{\max, t}.
\end{aligned}$$

Ini adalah problem yang secara konseptual sekarang ingin kita selesaikan.

AG.70. Tetapi jangan langsung coding formulasi sebesar ini

Ini penting.

Kita sebaiknya membangun secara bertahap:

AG-1

Binary selection:

$$x_i.$$

AG-2

Selection + covariance:

$$\mathbf{x}^\top \Sigma \mathbf{x}.$$

AG-3

Selection + factor exposure.

AG-4

Continuous sizing:

$$w_i.$$

AG-5

Transaction cost.

AG-6

CVaR.

AG-7

Dynamic rebalancing.

AG-8

QAOA benchmark.

Dengan demikian setiap komponen dapat diuji secara independen.

AG.71. Dan inilah keputusan desain yang saya sarankan

Untuk **versi pertama sistem nyata**, jangan langsung mencoba:

1000 *assets*.

Mulai dengan:

$$N = 10 - 30$$

opportunities.

Misalnya:

Crypto

BTC, ETH + 8–15 altcoins.

US equities

10–20 liquid stocks/ETFs.

Macro

Gold + USD proxy + equity index.

Kemudian baru:

$$N \rightarrow 50 \rightarrow 100 \rightarrow 500.$$

AG.72. Mengapa?

Karena tujuan pertama bukan:

maximize profit.

Tujuan pertama adalah:

validate the mathematical architecture.

Kita harus bisa membuktikan:

Prediction → QUBO → Selection → Sizing → Risk → Execution

benar-benar bekerja tanpa leakage dan tanpa bug.

AG.73. Dan ada satu konsekuensi besar dari AG

Mulai tahap ini, **backtesting** kita tidak boleh lagi hanya mengevaluasi signal.

Kita harus melakukan:

event – driven portfolio backtest.

Karena keputusan sekarang bergantung pada:

$x_t, w_t, w_{t-1}, fees, slippage, liquidity, risk, position\ state.$

Jadi backtest harus mengetahui **state portfolio**, bukan hanya:

BUY
SELL
HOLD

AG.74. State sistem

Pada waktu t , state minimal:

$$S_t = (Cash_t, w_t, Positions_t, Orders_t, SL_t, TP_t, Equity_t, Drawdown_t).$$

Kemudian market memberikan:

$$M_t.$$

Model menghasilkan:

$$A_t = \pi(S_t, M_t).$$

Dan:

$$S_{t+1} = F(S_t, M_t, A_t).$$

Ini sebenarnya sudah mulai menyerupai **stochastic control problem**.

AG.75. Dan itu akan menjadi jembatan kita ke Tahap berikutnya

Karena setelah kita mempunyai:

$$x_t$$

dan:

$$w_t,$$

pertanyaan berikutnya adalah:

Bagaimana optimizer memilih portfolio ketika kondisi pasar berubah secara terus-menerus dan posisi yang sudah ada memiliki state masing-masing?

Dengan kata lain:

$$\text{Static Optimization} \rightarrow \text{Dynamic Optimization.}$$

Dan di sinilah kita akan mulai memasukkan secara matematis:

$$\text{state, transition, reward, control, stochasticity, rebalancing.}$$

Itu akan menjadi fondasi yang sangat penting untuk **Tahap AH**, karena pada titik tersebut sistem kita tidak lagi sekadar mencari portfolio optimal pada satu timestamp, tetapi mulai memecahkan:

“Bagaimana portfolio optimal berevolusi terhadap waktu?”